

フェルミオン対応 実装解説(レビュー用ガイド)

対象: **fermion** ブランチ(develop からの差分 33 コミット)。コードレビューの下敷きとして、**アルゴリズムの考え方 → 実装の構造 → ファイル・コードの対応 → 各所の検証根拠**の順に説明する。規約の確定経緯は設計書 [docs/superpowers/specs/2026-08-15-fermion-m1-design.md](https://docs.superpowers/specs/2026-08-15-fermion-m1-design.md)(改訂1~4)が正。

0. 何をやっているのか(3行)

フェルミオンの反交換符号は、テンソルネットワークでは「脚の交差」に局在する。本実装は脚ごとの Z_2 パリティを持つ薄いラッパー **ftensor** を導入し、縮約・転置・分解のたびに交差符号を機械生成する。既存のテンプレートカーネル(simple update)はそのまま **ftensor** でインスタンス化するだけで済み、ボゾン経路のコードと golden は一切変わらない。

1. アルゴリズムの原理

1.1 なぜ符号が出るか

第二量子化の状態は生成演算子の順序付き積で定義される:

$$|n_1 \ n_2 \ \dots \ n_N\rangle = (c^\dagger_1)^{n_1} (c^\dagger_2)^{n_2} \dots (c^\dagger_N)^{n_N} |0\rangle$$

テンソルネットワークの縮約は、この積の中身を並べ替える操作に対応する。奇パリティ(= フェルミオン数が奇数)の対象を2つ入れ替えるたびに -1 が付く。これがすべての符号の源で、実装上は次の1本の規則に集約される:

軸の置換で順序が反転する脚対のうち、両方が奇である対の個数を k とすると符号は $(-1)^k$

要素ごとに見れば、**idx** における脚 a の parity と脚 b の parity の積で判定できる(**ftensor.hpp** の **transpose_sign**)。

1.2 Z_2 超選択則

物理的な状態・演算子は**全脚パリティの合計が偶**でなければならない(奇成分は非物理)。実装ではこれを不変条件として扱い、**parity_violation()** で検査する。初期化・分解・更新のすべてで偶が保たれることをテストで確認している。

1.3 なぜ「1サイト測定」と「2サイト測定」で扱いが違うか

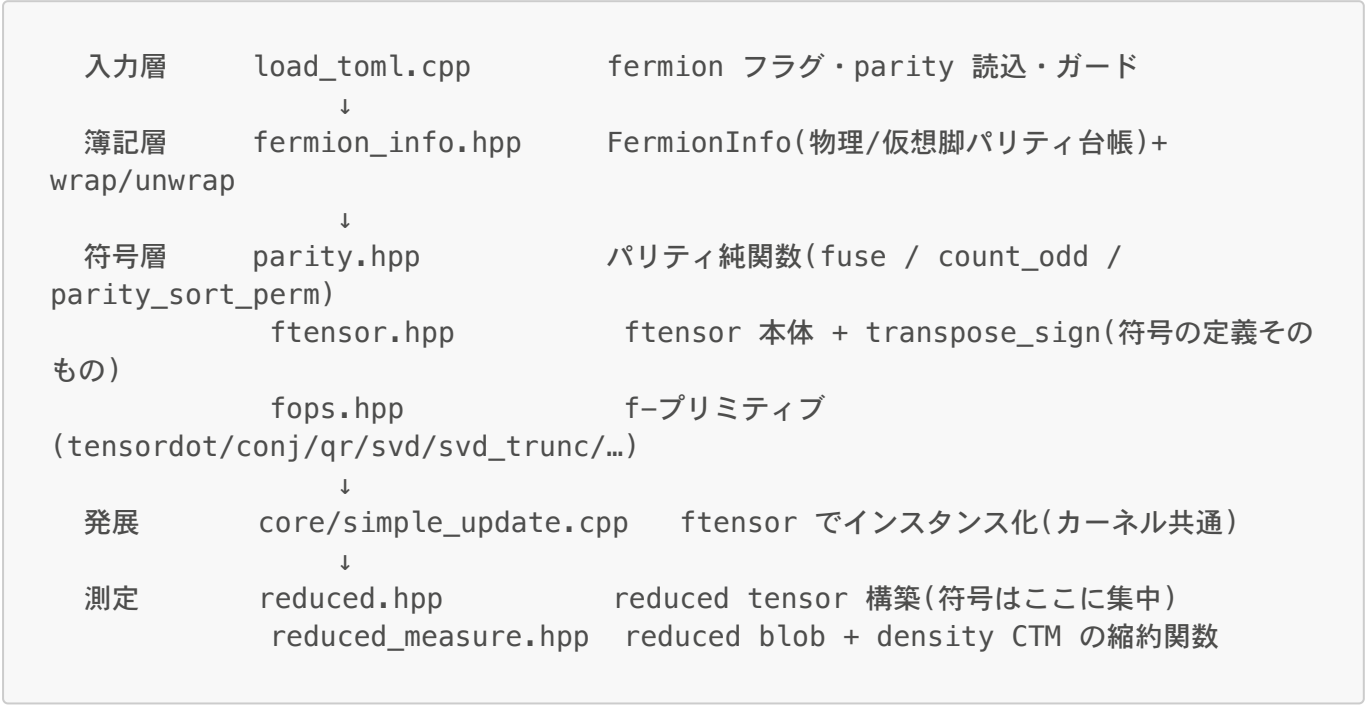
CTM 環境は**閉ループ**を含む。開いたネットワーク(状態の縮約)では上記の局所規則で完結するが、ループを閉じると supertrace(グレード付きトレース)の (-1) が混入し、素朴な機械簿記では規約が破綻する(設計書改訂1・2に、この破綻を実測して arrow 方式を破棄した経緯がある)。

そこで M1 の最終方針は:

- **simple update**(開ネットワーク)→ **ftensor** で直接計算

• **測定・環境**(ループあり)→ サイトごとに bra/ket 2層を融合した **reduced tensor** をまず構築し、そこから先は**交差の無い通常のテンソルネットワーク**として既存のボゾン用 density-CTM をそのまま使うつまり**フェルミオン符号を reduced tensor の構築関数1箇所に閉じ込める**。これが本実装の中心的な設計判断。

2. レイヤー構造とファイル対応



ファイル	行数	役割
src/fermion/parity.hpp	73	parity_vector / leg_parities 型と純関数3つ
src/fermion/ftensor.hpp	119	ftensor<tensor> 構造体、member transpose(符号適用の本体)、transpose_sign
src/fermion/fops.hpp	677	f-プリミティブ全部。mptensor と同名の自由関数でオーバーロード解決
src/fermion/fermion_info.hpp	95	FermionInfo(phys/virt パリティ)、wrap_Tn/unwrap_Tn、隣接整合検証
src/fermion/reduced.hpp	210	reduced tensor(1サイト・2サイト blob)の構築
src/fermion/reduced_measure.hpp	154	λ dressing、reduced 群の生成、blob と CTM 環境の縮約

3. 符号層の詳細(レビュー要点つき)

3.1 ftensor(ftensor.hpp)

```
template <class tensor>
struct ftensor {
    tensor t;                // 密テンソル本体(mptensor 型、無改造)
    leg_parities parity;    // parity[axis][index] = その添字のパリティ
};
```

パリティを「セクター次元 (D_e, D_o)」ではなく添字ごとの **bool** ベクトルで持つのが要点。reshape(脚の融合)後の非連続パターンや、CTM の **extend** によるパディングでも表現できる。

member **transpose** は「符号マスクを掛けてから **mptensor::transpose**」の順で、parity も同時に並べ替える。

レビュー観点: member と自由関数 **transpose** が同じ規約であること(自由関数は member に委譲)。

3.2 graded tensordot(**fops.hpp:264**)

意味論は「A の縮約脚を末尾へ、B の縮約脚を**逆順**で先頭へ寄せる」置換の符号 + 素の tensordot。実装は置換を実行せず**符号マスクだけ**を掛けて **mptensor::tensordot** に渡す(等価かつ高速):

```
a_masked = apply_transpose_sign_mask(a, tensordot_left_perm(...)); // (自由脚..., 縮約脚...)
b_masked = apply_transpose_sign_mask(b, tensordot_right_perm(...)); // (縮約脚逆順..., 自由脚...)
ret.t = mptensor::tensordot(a_masked.t, b_masked.t, axes_a, axes_b);
```

縮約脚対のパリティー一致は **validate_contracted_parity** が実行時検証する(規約ミスの早期検出)。

レビュー観点: **tensordot_right_perm** の逆順が B 側にのみ適用されていること。ここが規約の核。

3.3 conj(**fops.hpp:294**)

要素共役 + 要素ごとの符号 $(-1)^{\{m(m-1)/2\}}$ (m = その要素で奇な脚の数)。これは「全脚の順序反転」に相当する twist で、**Task 9 の JW 厳密参照テスト**が唯一の確定根拠。

3.4 graded QR / SVD(**fops.hpp:383 / :476 / :573**)

手順(QR も SVD も共通):

1. **transpose(a, rows + cols)** で行・列に整列(graded、符号込み)
2. 行・列を融合し **fuse** でパリティを計算
3. **parity_sort_perm** の置換行列との **tensordot** で偶先頭に整列(分散安全。直接インデックス書き込みをしない)
4. 偶ブロック・奇ブロックを **slice** で切り出し、各々 **mptensor::qr / svd**
5. 直和で組み直し、置換を戻す
6. 内部脚のパリティー(偶 k_e 個 + 奇 k_o 個)を返す

`svd_trunc(:573)`はさらに**セクター横断の切断**を行う: 全特異値を降順(同値なら偶優先)で dc 本選び、選択後に偶先頭へ再ソート。選択は**選択行列との tensordot** で実装(同じく分散安全)。

レビュー観点1: `validate_noninterleaved_split` — 行と列が交互(例 `rows=(0,2), cols=(1,3)`)だと Koszul マスクが行成分と列成分を結合する非分離 Hadamard 因子になり、**分解後の特異値が物理的な Schmidt スペクトルでなくなる**。これは実際にバグとして踏んだので、ガードで拒否している(呼ぶ側が非交差になるよう regroup する責任を負う。`core/simple_update.cpp` の `regroup_theta_for_svd`)。

レビュー観点2: 分散(ScaLAPACK)対応のため、要素ループでの直接書き込みを避けて置換行列/選択行列を使っている。`make_perm_matrix` の規約は $P[\text{new}, \text{old}] = \delta$ 。

4. simple update(発展)

4.1 カーネル(`src/iTPS/core/simple_update.cpp`)

ボゾンと完全に同一のソースを `ftensor` でインスタンス化する。カーネル側の変更は2点のみ:

1. 「svd → slice で切断」を `svd_trunc(...)` に置換(ボゾン用オーバーロードを `src/tensor.hpp` に追加して共通化)
2. `regroup_theta_for_svd` — θ を `(aux1, out1, aux2, out2)` に **graded transpose** してから SVD に渡す (§3.4 レビュー観点1 の非交差要件を満たすため。ここが素の transpose だと Koszul 符号が落ちる = 実際にあったバグ)

分解後に `enforce_even_parity` で超選択則を検査(閾値超過は例外、微小な数値ゴミは 0 にクリップ)。

4.2 ドライバ境界(`src/iTPS/simple_update.cpp:60-`)

`finfo.enabled` の分岐で `wrap` → カーネル → `unwrap` を行う。ここで**2つの規約**が入る:

```
// (a) ボンド方位の正規化: canonical はラスタ順(左→右、上→下)
if (source_leg == 0 || source_leg == 1) { // 右→左 / 下→上 で来た場合
    std::swap(s1, s2); std::swap(s1_leg, s2_leg);
    op12 = mptensor::transpose(up.op, Axes(1, 0, 3, 2));
}
// (b) ゲートの graded 表現: 入力脚の交差符号
auto fop = wrap_twosite_op(op12, finfo.phys[s1], finfo.phys[s2]);
```

- (a) がないと、JW 順で後のサイトからゲートを当てることになり誤ったマスクが掛かる (症状: 2D で市松 CDW への崩壊、D=4 で負ノルム)
- (b) がないと `|11⟩⟨11|` チャンnel(全 Trotter ゲートが持つ)が反転する (症状: τ^2 スケールの λ 乖離。ホッピングのみの参照テストでは検出できなかった)

レビュー観点: `wrap_twosite_op` は `apply_swap(fop, 0, 1)` のみ(入力脚だけ)。測定側の `wrap_reduced_pair_op` は入力・出力両方。この非対称性は §5.3 で説明する。

5. 測定(reduced tensor + density CTM)

5.1 考え方

サイトごとに bra 層と ket 層を融合した4本脚テンソル $a[(l \bar{l}), (t \bar{t}), (r \bar{r}), (b \bar{b})]$ を作る。この粗視化格子の上では交差が存在しないので、以降は既存のボゾン用 density-CTM 一式 (`Calc_CTM_Environment_density + contract_density_ctm/`) を無改造で使える。

5.2 構築パイプライン(`reduced.hpp`)

`doubled_pipeline(:92)` が共通の心臓部:

```
conj(bra) ⊗ ket          outer product(rank 10)
→ apply_joint_swaps     bra/ket 層をまたぐ交差の符号(下記)
→ (ket, bra) interleave graded transpose
→ column-major fusion   mptensor::reshape で [l l̄] 等に融合
```

`apply_joint_swaps` の `kDoubledJointMask` は「どの脚ペアに joint swap を入れるか」を固定した定数で、解析的に導出したものではなく、**Fock oracle** を審判にした探索で同定したもの(コメントに明記)。YASTN の `fuse_layers()` 規約とゲージ同値であることを確認している。

用途別の入口:

関数	用途
<code>build_reduced_op</code>	物理脚を開いたまま(1サイト演算子の挿入用)
<code>build_reduced</code>	物理脚をトレース(ノルム用)
<code>build_reduced_pair</code>	2サイト cluster を doubling して 6 本脚 blob(2サイト演算子用)

5.3 演算子ロードの3規約(最重要の注意点)

経路	規約	ヘルパ	根拠
simple update ゲート	入力脚 swap	<code>wrap_twosite_op</code>	d=2 判別テスト + λ 軌跡 + Fock 審判
2サイト測定 blob	入力 + 出力 swap	<code>wrap_reduced_pair_op</code>	R3(d=2)+ R5(d=4) oracle
1サイト(ゲート・測定)	素	—	入力脚の交差が存在しない

d=2 では blob の「素ロード」と「両swap」が**N保存演算子上で縮退**するため、当初 R3 は「素」と誤って固定していた。d=4(電子系)の (odd,odd)→(even,even) チャンネル(スピンありホッピングの線形項)で初めて分岐し、R5 で修正した。

レビュー観点: この3規約が混ざると符号が黙って狂う。新しい経路を足すときは必ず oracle テストを先に書く。

5.4 測定ドライバ(`src/iTPS/measure.cpp`, `onsite_obs.cpp`, `twosite_obs.cpp`)

- `measure.cpp:45-` : fermion 時、**bare Tn**(λ dressing なし)から reduced 群を作って density CTM を回す
 - TeNeS 規約ではカーネルが $\sqrt{\text{Schmidt}}$ を bond 両端の Tn に書き込むため、状態は bare Tn の直接縮約
 - full- λ dressing は MeanField 経路の規約。CTM に渡すと環境重みの二重計上になる(実際にバグだった)
- `onsite_obs.cpp:87-` : reduced impurity + `Contract_one_site_density_CTM`
- `twosite_obs.cpp:174-`(ノルム)、`:217-`(縦ボンド)、`:259-`(横ボンド): blob + 専用の窓関数 (`contract_reduced_pair_{horizontal,vertical}_density_CTM`)

6. 入力層とガード(`load_toml.cpp`)

- `[parameter.general]` fermion、`[[tensor.unitcell]]` parity の読み
- `validate_fermion_constraints(:602-)`が M1 の非対応機能を**入力時に**まとめて拒否 (非 ground state / full update / MeanField / Simple_Gauge_Fix / RSVD / 相関関数 / multisite / 距離2以上の2サイト / tensor_save・load / パリティ奇の演算子 / 奇プロダクト初期状態)
- 相関長のみ「デフォルト有効」なので、エラーではなく**強制無効化 + 警告**

レビュー観点:「沈黙して間違った数値を出さない」が原則。ガードの網羅性は仕様書 §6.1 の表と対照。

7. テストの地図(何がどの層を保証しているか)

層1: 単体(`test/test_fermion_layer.cpp` 前半、行番号は現在の値)

対象	テスト
parity 純関数	<code>fuse ...column-major with XOR(:95)</code> ほか
マスク演算	<code>apply_swap negates odd-odd elements only(:115)</code>
graded transpose	<code>applies Koszul signs(:154)</code> 、 <code>transpose twice returns original(:170)</code>
tensordot	<code>rejects parity mismatch(:197)</code> 、 <code>contraction-order independent(:207)</code> 、 <code>matches manual swap(:228)</code>
conj	<code>norm ... positive via conj(:260)</code> 、 <code>involution(:279)</code>
QR/SVD	<code>graded QR reconstructs(:418)</code> 、 <code>graded SVD reconstructs(:445)</code> 、 <code>svd_trunc selects across sectors(:468)</code> 、 <code>rejects interleaved row-column splits(:736)</code>

層2: 規約固定(厳密参照との対査)

テスト	何を固定するか
<code>JW four-site reference matches f-primitive contractions(:588)</code>	<code>conj</code> と <code>tensordot</code> の規約(JW 厳密値、 <code>rtol 1e-12</code>)

テスト	何を固定するか
<code>two-site operator doubly-odd input channel ...(:636)</code>	<code>wrap_twosite_op</code> (ゲート規約)
<code>manual four-swap reduced tensor ...(:672)</code>	文献規約(Corboz)との橋渡し
<code>r2_convention.cpp</code> R2/R3	Fock oracle vs f-プリミティブ vs reduced(d=2、3者一致)
<code>r2_convention.cpp</code> R5(:619)	d=4 の blob 演算子規約 (Fock 検証済み直接経路が審判)

層3: 統合(常時実行)

- `reduced density CTM measurement gives positive fermion norms`(:764)
- `fermion twosite measurement is translation invariant across wraps`(:3027)
- `ctest FreeFermion`: E2E(既定は D=2 のみの quick モード、48秒)

層4: 診断(環境変数ゲート、常時は skip)

環境変数	用途
<code>TENES_RUN_LAMBDA_TRAJECTORY_DIAG</code>	横鎖入軌跡を JW 双子ボゾンと比較(case=A/B 判定)
<code>TENES_RUN_VERTICAL_LAMBDA_DIAG</code>	同・縦鎖(leg 1/3 で向き規約を判別)
<code>TENES_RUN_PLAQUETTE_TROTTER_DIAG</code>	開放 2x2 パッチでカーネル vs 厳密 Trotter vs Fock(3者)
<code>TENES_RUN_WEAK2D_DIAG</code>	弱2D結合で CTM 測定 vs CTM 非依存の平均場推定(測定バグの切り分け)
<code>TENES_RUN_ELECTRON_CHAIN_DIAG</code>	d=4 電子鎖(θ 成分ダンプ、Fock 審判用)

注意: JW 双子入判定は **d=2 でのみ有効**。d=4 では奇セクター多重度が 2 になり成分の素コピーが JW 対応でなくなるため、case=B が出てもバグの証拠にならない(設計書 改訂4)。

8. レビューで特に見てほしい箇所

1. `reduced.hpp` の `kDoubledJointMask` と `apply_fused_leg_gauge` — 唯一「解析導出でなく実験的に固定した」部分。oracle テストで縛ってはいるが、導出を与えられるならそれが理想
2. **演算子ロードの3規約**(§5.3) — 経路ごとに違うのは設計として気持ち悪い。統一できないか
3. `svd_trunc` の切断規約 — セクター横断の降順選択が変分的に正しいか(縮退時の偶優先は決定論性のため)
4. `validate_noninterleaved_split` の責任分界 — ガードで拒否 + 呼び出し側が regroup、で妥当か
5. **性能**: `doubled_cluster` が D^{12} スケーリング(D=4 の2サイト測定に6時間超)。改善案は設計書の性能課題節に記載(bra 要素ループ + ket ベクトル演算で ~100x 見込み)
6. `iTPS.hpp` の `finfo` メンバと分岐の入り方 — ボゾン経路への影響がゼロであることの確認

9. 既知の制限(M1 スコープ外)

- expert モードのみ(`tenes_simple/tenes_std` は未対応)
- 距離2以上の観測量・相関関数・相関長・有限温度・実時間発展・full update・MeanField 環境
- `tensor_save/tensor_load`(FermionInfo の直列化が未実装)
- $n \neq 0.5$ ($\mu \neq 0$)では simple update の収束が大幅に遅い(必要虚時間 ~45 vs 半充填 ~10)
- $d=4$ (電子系)は規約固定済み・1D 極限検証済みだが、**2D の量的検証は未実施**